1 · Il progetto in un minuto

Se devi dirlo in 30 secondi
È un **chatbot da terminale** in Python che parla con un **modello locale** (Gemma 4 in LM Studio). Il flusso è un **grafo LangGraph** esplicito: un nodo chiama il modello, un nodo esegue gli strumenti, una funzione decide dove andare. Quando chiedi l'ora il modello **non la inventa**: chiede lo strumento **orologio**, il programma lo esegue davvero e il risultato torna al modello. La cronologia dura per la sessione, si avvia anche con **Docker Compose** e ci sono **70 test** che non richiedono il modello.

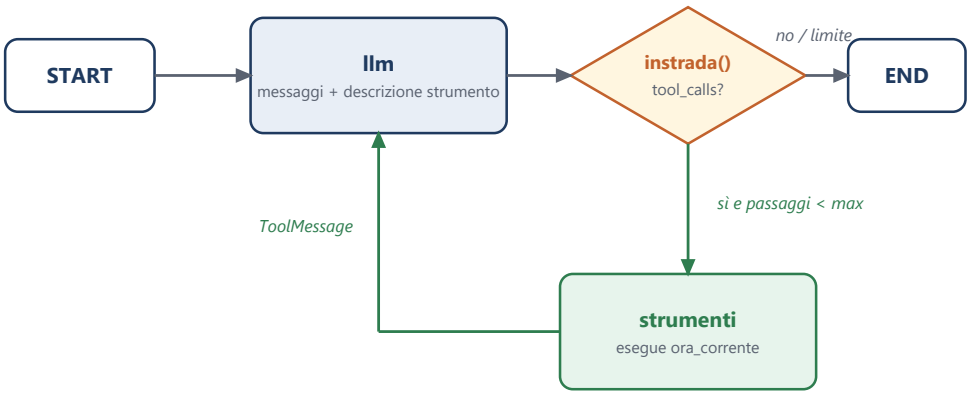
Le cinque idee da far passare

- **Il modello è senza stato**: la memoria è dell'applicazione, che reinvia la cronologia a ogni turno.
- **Il modello chiede, il programma esegue**: il tool calling è una richiesta strutturata, non un'esecuzione.
- **Grafo esplicito**: stato, nodi e instradamento si vedono, si testano e si estendono.
- **Nessuna risposta inventata**: errori chiari, limite di passaggi, rollback del turno.
- **Verificato davvero**: test senza modello + prove reali con LM Studio e Docker.

Numeri da ricordare

- Modello `google/gemma-4-e2b`, GGUF Q4_K_M, contesto 8192 token.
- Python 3.13 · LangGraph 1.2.18 · langchain-openai 1.7.3.
- 7 moduli, 467 righe di codice.
- **70** test automatici + **3** prove reali LM Studio + **1** prova Docker.
- Tool calling corretto: da **12/17** a **12/12** dopo aver migliorato la descrizione.
- `MAX_PASSAGGI` predefinito: 6 chiamate al modello per turno.

Il grafo: lo schema da disegnare alla lavagna



START → llm → (strumenti → llm) → END. Si esce quando il modello non chiede più strumenti oppure al limite di passaggi.* `instrada()` *è scritta nel progetto, così la decisione è leggibile e finisce nei log.*

Come appare una sessione (prova reale del 28 ottobre)

```
Tu> Che ore sono adesso a Tokyo?
[strumento] ora_corrente(fuso_orario="Asia/Tokyo") → 2026-10-28T20:08:51+09:00 (Asia/Tokyo), mercoledì
Assistente> Adesso sono le 20:08:51 di mercoledì 28 ottobre 2026 a Tokyo.
Tu> Che giorno della settimana è oggi a New York?
[strumento] ora_corrente(fuso_orario="America/New_York")
            → 2026-10-28T07:09:08-04:00 (America/New_York), mercoledì
Assistente> Oggi a New York è mercoledì.
```

Requisiti della consegna (slide 26)

Requisito	Evidenza	Stato
Grafo con LLM e cronologia	Al secondo turno il modello ricorda «Ada» (prova reale)	Fatto
Strumento orologio	Riga [strumento] in chat e log INFO di richiesta e risultato	Fatto
Docker Compose	Prova -m docker superata; chat interattiva provata a mano	Fatto
Test e documentazione	70 automatici + prove reali; README, AVVIO.md e CLAUDE.md	Fatto
Repository condivisa	Conventional Commits in italiano; push su origin/main	Fatto

2 · La teoria essenziale

LLM locale e LM Studio

- Un LLM prevede il **token successivo**. Ne derivano tre proprietà: è **senza stato** (ogni chiamata è indipendente, la cronologia va reinviata), **non ha accesso al mondo** (non legge l'orologio: serve uno strumento), è **probabilistico** (le prove reali vanno ripetute e contate, es. 12 su 12).
- **LM Studio** esegue modelli GGUF sul PC e li espone con un'**API compatibile con OpenAI** su `http://localhost:1234/v1`. Il progetto usa `GET /v1/models` all'avvio (il modello configurato esiste?) e `POST /v1/chat/completions` a ogni chiamata. Client: **ChatOpenAI** con un altro `base_url`.
- **Quantizzazione Q4_K_M**: pesi a circa 4 bit invece di 16, così il modello gira su un PC con poca perdita di qualità.
- Ruoli dei messaggi: **system** (istruzioni), **user** (domanda), **assistant** (risposta o `tool_calls`), **tool** (risultato, legato alla richiesta dal `tool_call_id`).

LangGraph: il flusso diventa un grafo

Elemento	Che cos'è	Nel progetto
Stato	Dati condivisi durante l'esecuzione (TypedDict)	`messages`, `passaggi`, `limite_raggiunto`
Nodo	Funzione: legge lo stato, restituisce un aggiornamento parziale	`llm` e `strumenti`
Arco / arco condizionale	Collegamento fisso / scelto da una funzione sullo stato	START → llm, strumenti → llm / `instrada()`
Reducer	Regola che combina l'aggiornamento con lo stato	`add_messages`: **aggiunge** i messaggi, non sostituisce la lista
compile / invoke	Grafo eseguibile / esecuzione che restituisce lo stato finale	`stream(stream_mode="updates")` nei test per vedere i nodi
Ciclo	Arco che torna indietro: base degli **agenti**	strumenti → llm, con uscita garantita dal limite

Tool calling: il modello chiede, il programma esegue

Passo	Chi	Che cosa succede
1	Programma	Invia i messaggi e, nel campo `tools`, lo strumento descritto in **JSON Schema** (nome, descrizione, parametri).
2	Modello	Se serve risponde con `tool_calls`: nome, argomenti JSON e un id (es. `call_1`).
3	Programma	Il nodo strumenti esegue la funzione reale e aggiunge un **ToolMessage** con lo stesso `tool_call_id`.
4	Modello	Legge il risultato e risponde (o chiede un altro strumento).

La descrizione è il contratto
È l'unica cosa che il modello sa dello strumento. Senza la descrizione del parametro il modello ometteva `fuso_orario` in 5 casi su 17 (e dava l'ora di Roma); con descrizione del parametro e una frase nel prompt: 12 su 12.

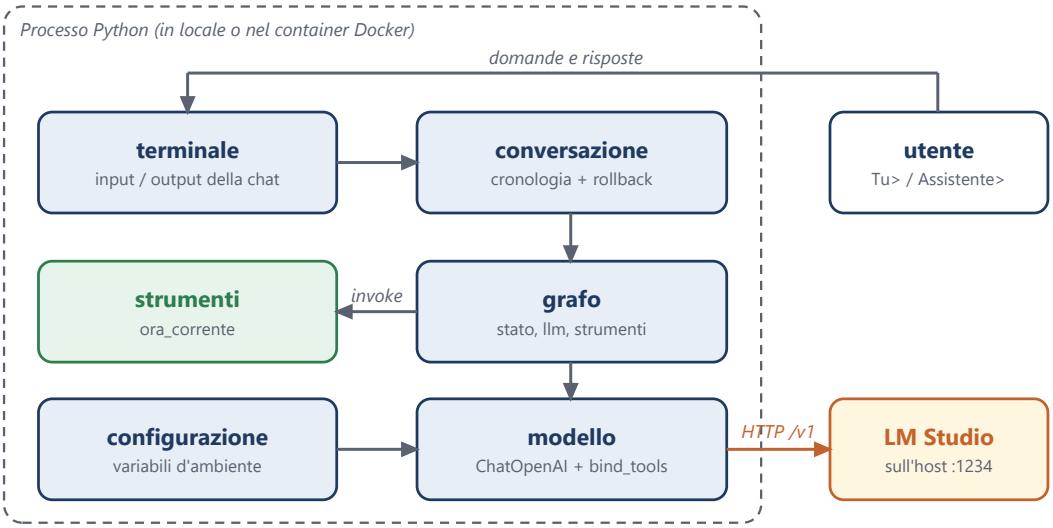
Tempo e fusi orari

- L'istante si legge **sempre in UTC** e si converte nel fuso richiesto: il fuso del PC o del container non conta.
- **Fuso IANA** (`Europe/Rome`, `Asia/Tokyo`): conosce l'ora legale; un offset fisso no.
- **ISO 8601**: `2026-10-28T20:08:51+09:00`.
- **zoneinfo** + **tzdata** (database dei fusi, assente su Windows e nell'immagine slim).
- Giorno della settimana calcolato nel fuso richiesto, con nomi italiani da tabella.

Docker e Docker Compose

- **Immagine** = pacchetto immutabile; **container** = istanza in esecuzione.
- Dockerfile: `python:3.13-slim`, prima i requirements poi il codice (cache dei layer).
- Il container raggiunge LM Studio su `host.docker.internal:1234`: nel container `localhost` è il container stesso.
- `environment` prevale su `env_file`; `stdin_open` + `tty` per la chat.
- Chat con `docker compose run`, non up. LM Studio **non** va containerizzato.

3 · Come è fatto



__main__ legge la configurazione, crea strumento, modello, grafo e conversazione, poi avvia la chat

Moduli piccoli, una responsabilità ciascuno. Le lezioni successive estenderanno questo codice: un nuovo strumento è un altro BaseTool aggiunto alla lista `strumenti`.

Modulo	Responsabilità
`configurazione.py`	Legge e valida le variabili d'ambiente; l'errore nomina la variabile.
`strumenti.py`	`ora_corrente(fuso_orario?)`: validazione del fuso IANA e formato del risultato.
`modello.py`	ChatOpenAI verso LM Studio con `bind_tools`; controllo di GET /v1/models.
`grafo.py`	Stato, nodi llm e strumenti, `instrada`, limite di passaggi, log.
`conversazione.py`	Cronologia di sessione, rollback del turno, errori del modello.
`terminale.py · __main__.py`	Ciclo Tu> / Assistente>, righe [strumento]; avvio e composizione dei pezzi.

Un turno passo per passo: «Che ore sono adesso a Tokyo?»

#	Messaggio	Prodotto da	Contenuto
1	HumanMessage	Conversazione	Che ore sono adesso a Tokyo?
2	AIMessage	nodo llm (passaggio 1)	testo vuoto + `tool_calls`: ora_corrente({"fuso_orario": "Asia/Tokyo"}), id call_...
3	ToolMessage	nodo strumenti	2026-10-28T20:08:51+09:00 (Asia/Tokyo), mercoledì · stesso tool_call_id
4	AIMessage	nodo llm (passaggio 2)	Adesso sono le 20:08:51 di mercoledì 28 ottobre 2026 a Tokyo.

Con `LIVELLO_LOG=INFO` i log mostrano: inizio e fine di ogni chiamata al modello con la durata, la decisione di **instradamento**, la richiesta di strumento con gli argomenti e il risultato. Alla domanda successiva «E a Roma?» il modello passa `Europe/Rome`: ha capito il riferimento grazie alla cronologia.

Cronologia e rollback

- **Conversazione** invoca il grafo con `[*cronologia, HumanMessage(testo)]` e `passaggi=0`.
- La cronologia viene sostituita **solo se il turno riesce**.
- Se il turno fallisce torna esattamente com'era: niente domanda orfana, niente richieste di tool senza risposta.
- **Niente checkpointer**: la persistenza non è richiesta e il rollback resta semplice.

Errori: mai una risposta inventata

- LM Studio spento: avviso all'avvio, errore chiaro in chat, sessione aperta.
- Modello inesistente o configurazione errata: uscita con codice 1 e messaggio chiaro, senza traceback.
- Strumento sconosciuto, argomenti errati o non JSON: ToolMessage che inizia con `Errore:`.
- Limite di passaggi (3 llm, 2 strumenti con MAX=3): messaggio esplicito + rollback.
- Ctrl+C, fine input, «esci»: uscita pulita.

4 · Test, metodo e risultati

Due livelli di verifica

Livello	Che cosa prova	Comando · esito
Test automatici (70)	Il **programma**: percorsi del grafo, ToolMessage, limite, rollback, log, orologio con istante fisso (Roma inverno/estate, Tokyo, UTC, cambio di giorno), configurazione, terminale, avvio.	`python -m pytest` 70 passed, 4 deselected
Prove reali LM Studio (3)	Il **modello**: ricorda «Ada» al secondo turno; chiede davvero l'orologio per Tokyo e usa l'ora restituita; non usa lo strumento per una domanda ordinaria.	`python -m pytest -m lmstudio -v` 3 passed
Prova Docker (1)	Dal container: riga [strumento] con Asia/Tokyo e risposta che usa quell'ora.	`docker compose build`, poi `python -m pytest -m docker -v` 1 passed

- **Modelli finti** (`tests/finti.py`): GenericFakeChatModel e un RunnableLambda «a copione» restituiscono testo, richieste di tool o eccezioni: si provano tutti i percorsi senza LM Studio.
- **Dipendenze iniettabili**: l'orologio (`orologio=lambda: ISTANTE_INVERNO`) e le funzioni leggi/scrivi del terminale.
- **TDD**: ogni funzionalità e ogni correzione è partita da un test visto fallire (RED → GREEN).
- Una risposta simulata **non** dimostra che il modello sappia usare lo strumento: per questo esistono le prove reali.

Metodo di lavoro

Fase (Superpowers)	Che cosa è stato prodotto
CLAUDE.md	Istruzioni stabili per Claude Code: obiettivo, confini, scelte tecniche, test, regole Git.
Brainstorming → specifica	Decisioni con lo studente; specifica e requisiti con ID (RF, RE, RV) in docs/superpowers.
Piano → implementazione	Cinque incrementi con test da scrivere prima; un commit per incremento verificato.
Revisione indipendente	Nessun problema critico; 5 problemi importanti corretti partendo da un test che falliva.
Git	Conventional Commits in italiano (`feat(grafo): …`, `fix(avvio): …`), staging selettivo, push su richiesta.

Risultati e decisioni

Versione dello strumento	Ora a Tokyo con fuso giusto	Domanda senza strumento
Senza descrizione del parametro	12 su 17 (5 volte senza fuso: ora di Roma)	9 su 9
Con descrizione del parametro + frase nel prompt	**12 su 12**	**12 su 12**

- Il modello non ha mai inventato l'ora: mancava l'argomento. **Il test non è stato ammorbidito, è stato migliorato il contratto.**
- Giorno della settimana: il modello aveva scritto «lunedì» (era mercoledì) → ora il giorno è nel risultato dello strumento.
- **Lentezza**: Gemma 4 genera un **ragionamento nascosto** di 140–195 token (circa il 90% dei token) a circa 16 token/s. Con `reasoning_effort="none"` una risposta scende da 11,9 s a 3,6 s, ma non è stato adottato: l'affidabilità 12/12 va rimisurata prima.

Limiti e punti aperti (dirli prima che li chiedano)

- LM Studio spento e Ctrl+C dal vivo coperti dai test ma non provati a mano.
- La prova Docker non ricostruisce l'immagine: prima va fatto docker compose build.

5 · L'esposizione: scaletta, demo, domande

Tempo	Parte	Messaggio chiave
1 min	Obiettivo	Chatbot locale con grafo esplicito, orologio reale, Docker; requisiti della slide 26.
2 min	LangGraph	Stato, nodi, archi, add_messages; dal grafo fisso al ciclo con strumenti (schema di pagina 1).
2 min	Tool calling	Il modello chiede, il programma esegue; la descrizione è il contratto; tool_call_id.
1 min	Architettura	Moduli piccoli e separati; LM Studio sull'host, app nel container.
3 min	Demo	Domanda ordinaria, domanda sull'ora con riga [strumento] e log, avvio in Docker.
2 min	Test e metodo	70 test senza modello, prove reali separate, TDD, Superpowers, Conventional Commits.
1 min	Risultati e limiti	12/17 → 12/12, giorno della settimana, ragionamento nascosto, punti aperti.

```
.\.venv\Scripts\Activate.ps1 ; $env:LIVELLO_LOG="INFO" ; python -m assistente
Tu> Qual è la capitale della Francia?    → nessuna riga [strumento]
Tu> Che ore sono adesso a Tokyo?         → riga [strumento] + log di instradamento
Tu> E a Roma?                            → la cronologia fa capire il riferimento
docker compose build ; docker compose run --rm --name assistente-demo -e LIVELLO_LOG=INFO assistente
docker logs -f assistente-demo           (da un secondo terminale) ;  python -m pytest
```

Domande probabili: risposte in una riga

Domanda	Risposta
Perché LangGraph e non un ciclo while?	Il flusso diventa esplicito e verificabile: i test controllano quali nodi girano e aggiungere strumenti non richiede di riscrivere il ciclo.
Il modello esegue l'orologio?	No: produce solo una richiesta (nome + argomenti JSON). La funzione la esegue il nodo strumenti e il risultato torna come ToolMessage.
Come sapete che l'ora non è inventata?	Riga [strumento] e log con richiesta, esecuzione e risultato; il test reale controlla che la risposta contenga l'HH:MM dello strumento.
Dove sta la memoria?	Nell'applicazione: Conversazione tiene i messaggi e li reinvia; add_messages aggiunge senza sostituire. Alla chiusura si perde.
Se LM Studio è spento?	Avviso all'avvio, errore chiaro in chat, sessione aperta e cronologia com'era prima della domanda.
A che serve il limite di passaggi?	Ferma un modello che chiede lo strumento all'infinito: messaggio esplicito e rollback, nessuna risposta inventata.
Perché l'ora non dipende dal container?	L'istante si legge in UTC e si converte con zoneinfo + tzdata: il container era in UTC ma lo strumento dava l'ora giusta di Tokyo.
Come parla il container con LM Studio?	Connessione in uscita verso host.docker.internal:1234; nessuna porta pubblicata, localhost sarebbe il container.
Come si testa un grafo senza modello?	Modelli finti con risposte prestabilite (testo, tool_calls, eccezioni) e un istante fisso per l'orologio.
Perché è lento?	Ragionamento nascosto (~90% dei token) a ~16 token/s e due chiamate per le domande sull'ora.
Come si aggiunge uno strumento?	Un altro BaseTool nella lista strumenti passata a crea_modello e crea_grafo: il nodo lo trova per nome.

Prima di iniziare
Avvia LM Studio e carica il modello prima (la prima chiamata è più lenta). Avvisa che ogni risposta richiede qualche secondo e spiega perché. Tieni aperta una traccia di log già ottenuta, nel caso il modello rallenti.